УНИВЕРСИТЕТ ИТМО

Лабораторная работа №2

Реализация микросервисов

Выполнил:	Камалов Руслан Олегович
Группа:	БР1.2
ИСУ:	408732

Санкт-Петербург

2026

Отчёт по лабораторной работе №2

Студент: Камалов Руслан Олегович · Группа: БР1.2 · ИСУ: 408732

Цель

Я разделил монолит FitFlow на четыре независимо запускаемых микросервиса.

Реализация

`auth-service` владеет пользователями и авторизацией, `workout-service` - каталогом и планами, `progress-service` - фактами выполнения и проекцией тренировок, `content-service` - статьями. Каждый сервис имеет собственную точку входа, liveness health-check и свою базу. Основные префиксы API сохранены, но версии не полностью эквивалентны: в сервисах нет PATCH `/users/me` и фильтра maxDuration; в progress поле completedAt обязательно и не допускает будущую дату. Postman-сценарий нужно сверять с этими различиями, одной замены baseUrl недостаточно для произвольного сценария.

Точки входа расположены в `services/auth`, `services/workout`, `services/progress` и `services/content`; локальные порты соответственно 3001, 3002, 3003 и 3004. Nginx предоставляет единый внешний префикс `/api/v1`.

Проверка и вывод

Я проверил компиляцию всего workspace и корректность маршрутизации gateway статическими тестами. Затем я запустил четыре сервиса на отдельных локальных PostgreSQL БД и выполнил сквозной HTTP-сценарий напрямую через их порты. Все четыре health-check вернули `ok`; были успешно созданы пользователь, тренировка, план, запись прогресса и статья. Сводка вернула `completedWorkouts=1` и `totalMinutes=35`. Контейнерный запуск через gateway подготовлен в Compose и выполняется при включённом Docker daemon. Разделение уменьшило связанность доменов и позволило масштабировать их независимо.

Приложение

ДЗ 4. Архитектура микросервисов FitFlow

Декомпозиция

Сервис	Ответственность	Собственная БД
`auth-service`	регистрация, вход, профиль, роли, выпуск JWT	`auth_db`: `users`
`workout-service`	каталог тренировок и личные планы	`workout_db`: `workouts`, `training_plans`, `plan_workouts`
`progress-service`	история выполнений, агрегаты и проекция тренировок	`progress_db`: `progress_entries`, `workout_snapshots`
`content-service`	статьи	`content_db`: `articles`

Я применил database-per-service: сервис не читает таблицы другого сервиса и не использует межбазовые внешние ключи. `userId` и `workoutId` в чужой БД - логические UUID. Имена в таблице обозначают владельцев; в Compose каждый отдельный PostgreSQL-контейнер содержит базу `fitflow`.

Синхронные взаимодействия

Клиент обращается к сервисам через Nginx/API Gateway с единым внешним префиксом `/api/v1`:

- `/api/v1/auth/*` → auth;
- `/api/v1/workouts/*`, `/api/v1/plans/*` → workout;
- `/api/v1/progress/*` → progress;
- `/api/v1/articles/*` → content.

Каждый сервис самостоятельно проверяет подпись JWT общим симметричным секретом `JWT_SECRET`. Асимметричная подпись и JWKS - возможное развитие, не текущая реализация.

Асинхронные взаимодействия

RabbitMQ exchange `fitflow.events` типа `topic`, durable messages:

Событие	Издатель	Потребитель	Назначение
`workout.created`	workout	progress	обновление локальной проекции допустимых тренировок

Конверт события: `id`, `type`, `occurredAt`, `payload`. Я реализовал только `workout.created`. Потребитель делает upsert по ID тренировки и сохраняет `sourceEventId`; отдельного

журнала дедупликации по `event.id` нет. После записи он отправляет ack, при ошибке - nack без повторной постановки. Retry, DLQ, transactional outbox и publisher confirms не реализованы. События удаления пользователей и тренировок остаются предложениями по развитию.

```
flowchart LR
  C[Client] --> N[Nginx / API Gateway]
  N --> A[Auth service]
  N --> W[Workout service]
  N --> P[Progress service]
  N --> T[Content service]
  A --> ADB[(auth_db)]
  W --> WDB[(workout_db)]
  P --> PDB[(progress_db)]
  T --> TDB[(content_db)]
  W --> MQ[(RabbitMQ)]
  MQ --> P
```

Отказоустойчивость и согласованность

- локальная транзакция ограничена одной БД;
- межсервисные изменения имеют eventual consistency;
- `/health` сообщает только liveness; проверки readiness БД/RabbitMQ нет;
- запись тренировки и отправка события не атомарны: при сбое между операциями событие может потеряться;
- автоматического reconnect, retry и DLQ нет;
- проекция обновляется только при создании, изменения и удаления не синхронизируются;
- проверка проекции включается через `REQUIRE_KNOWN_WORKOUT=true`; текущий Compose её не включает.

Безопасность

Я передаю секреты через окружение. Node.js-приложения в образе работают без root, административные маршруты проверяют роль, порты БД не публикуются наружу. Используется console; структурированные логи, correlation ID и систематическое маскирование чувствительных полей пока не реализованы. Учебные секреты по умолчанию необходимо заменить перед эксплуатацией.

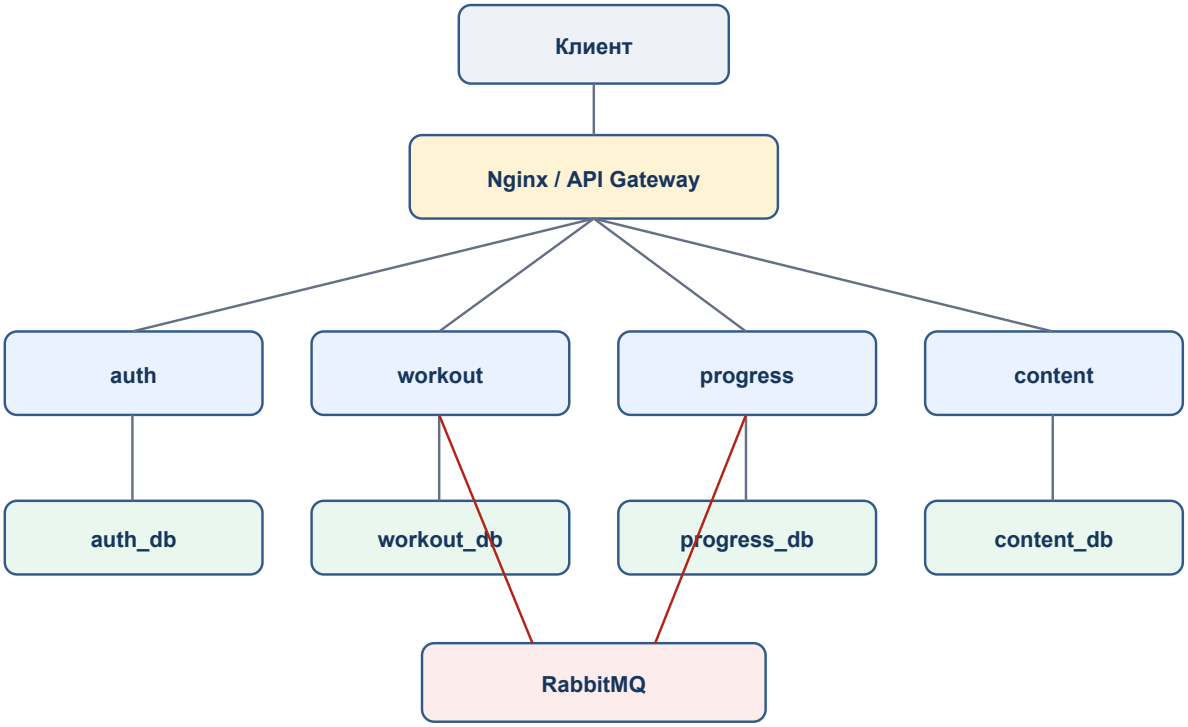


Рисунок 1 — микросервисная архитектура FitFlow